Laboratorio II – 1° modulo

Lezione 6

Lettura e scrittura di file di dati

Indice

- I `namespace`
  - `mystd::cout`
- Lettura e scrittura di file di dati
- Esercizi
- **Approfondimenti**
  - Algoritmi di *sorting*
  - Smart pointer
  - `std::stringstream`
  - L'attributo `auto`
  - *Function* pointer
  - Funtori (i.e. l'overloading dell'operatore `()`)

I namespace

I **namespace** permettono di raggruppare varie entità sotto un unico nome:

```
namespace nomespazio
  {
    int a;
    double b;
  }
```

Nella prima lezione è stato visto come è possibile definire e raggruppare le costanti usate nel codice tramite la direttiva al preprocessore: `#define`

In questo modo non è però possibile specificare il tipo della costante (viene determinato dal compilatore), le costanti cosi definite non sono *type safe* → è più opportuno quindi usare i `namespace`, cioè immergere le costanti in un `namespace`, per esempio definito prima del `main`

```
namespace CONST
{
  int    a = 10;
  double b = 3.14;
}
```

Utilizzo: `cout << CONST::a ...`

- Per utilizzarli: `nomespazio::a`
  `nomespazio::b`
- Proprio come il `namespace` "`std`":
  `std::cout << "ciao" << std::endl;`
- L'uso dei `namespace` permette di evitare conflitti tra nomi di identificatori (variabili, funzioni, strutture, etc…), soprattutto in codici di grandi dimensioni e complessi. Per esempio se si hanno funzioni che posseggono lo stesso nome ma che operano su dati diversi, è possibile, per leggibilità del codice, immergerle nei rispettivi `namespace`. Si veda il semplice esempio seguente

I namespace

Si vedrà ora un semplice esempio in cui verranno **immerse** delle **funzioni** di una **libraria** in un **namespace**

Nel file **myLib.h** si scriveranno come al solito i prototipi:

```
#ifndef MYLIB_H
#define MYLIB_H

#include <iostream>

namespace Int
{
  int Add (int a, int b);
  void Print (int a);
}

namespace Double
{
  double Add (double a, double b);
  void Print (double a);
}
#endif
```

Nel file **myLib.cc** andranno invece scritte le implementazioni:

```
#include "myLib.h"
using namespace std;

int Int::Add (int a, int b)
{
  cout << __PRETTY_FUNCTION__ << endl;
  return a + b;
}
void Int::Print (int a)
{
  cout << __PRETTY_FUNCTION__ << endl;
  cout << "Number: " << a << endl;
}

double Double::Add (double a, double b)
{
  cout << __PRETTY_FUNCTION__ << endl;
  return a + b;
}
void Double::Print(double a)
{
  cout << __PRETTY_FUNCTION__ << endl;
  cout << "Number: " << a << endl;
}
```

I namespace

Nel `main` program le funzioni verranno usate nel modo seguente:

```
#include "myLib.h"
using namespace std;

int main()
{
  cout << "Add two integers: " << Int::Add(2,3) << endl;

  Int::Print(10);

  cout << "Add two doubles: " << Double::Add(2.2,3.3) << endl;

  Double::Print(3.14);

  return 0;
}
```

Si può omettere il riferimento al `namespace` attraverso l'istruzione `using namespace`, da riportare prima dell'inizio del `main`, e.g. **`using namespace std;`**

N.B.: la `using namespace` **NON** va **MAI** utilizzata nello ***header*** file perché se lo *header* file venisse incluso in altre librerie potrebbe creare un conflitto tra i nomi delle funzioni/variabili. **E.g.:** se si scrive nello *header* file `using namespace CONST;` e lo si include in una libraria in cui vengono definite le variabili `a` e `b`, si ottiene un errore di compilazione

mystd::cout

Verrà illustrato ora un esempio semplice nel quale si realizzerà la `std::cout`, che verrà chiamata **`mystd::cout`**. Come si vedrà sarà necessario fare uso dei namespace, dell'operator overloading e dei template.
L'implementazione di `mystd::cout` non contemplerà la stampa a schermo, perché richiede dei dettagli tecnici nei quali non è necessario entrare, si cercherà però di riprodurre le essenziali caratteristiche della `std::cout`:

- La `cout` è immersa del **namespace `std`**
- La `cout` effettua l'**overload** dell'**operatore `<<`**
- L'operatore `<<` agisce su un qualunque tipo di dato alla sua destra (senza che il tipo sia specificato esplicitamente tramite la sintassi `<tipo>`)

E` necessario quindi creare la classe **`mycout`** che implementa l'overload dell'operatore `<<`, inoltre l'overload deve essere templetizzato:

```
class mycout {
public:
  template<typename T>
  mycout& operator<< (const T& buffer) {
    printf("Argument of size: %lu\n",sizeof(buffer));
    return *this; // Consente di concatenare: << ... << ... <<
  }
};
```

E` stato omesso il costruttore perché non è necessario

mystd::cout

A questo punto è necessario creare un oggetto di tipo `mycout` ed immergerlo in un namespace, che chiameremo **`mystd`**:

```
namespace mystd
{
  mycout cout;
}
```

Nel `main` verrà quindi usata proprio come la `std::cout` (è possibile evitare di esplicitare il tipo dell'argomento di `<<` grazie alla template argument deduction, *cfr. Approfondimenti Lezione 5*):

```
int main()
{
  mystd::cout << "Hello: " << 10 << 3.14;
  return 0;
}
```

Come è già stato detto l'intenzione non era quella di riprodurre la funzionalità della `std::cout`, cioè la stampa a schermo, si volevano però utilizzare tutti gli ingredienti propri della `std::cout` per realizzare una funzione simile

Input/Output in C++

- Sin dalla prima lezione è stata utilizzata la libreria standard `iostream` che ha permesso di acquisire o scrivere sequenzialmente messaggi o dati attraverso il terminale

- Queste informazioni vengono fornite da riga di comando dall'utente e si perdono non appena il terminale o l'eseguibile viene chiuso

- Volendo conservare i risultati prodotti per un uso successivo, risulta necessario scrivere fisicamente (salvare) queste informazioni volatili su di un file (file di caratteri, file binario, immagine, etc…)

- I comandi di `I/O` su file sono analoghi a quelli su terminale. Bisogna includere la libreria **`<fstream>`** che contiene le classi per l'`I/O` su file: www.cplusplus.com/reference/fstream/fstream

Lettura e scrittura da file

```
#include <iostream>
#include <fstream>
#include <string>
int main ()
{
  const char* filename = "File.txt";
  // Costruisce un oggetto di tipo ofstream passando
  // come parametro di input il nome del file
  std::ofstream OutFile(filename);

  int a         = 12;
  double b      = 24.5;
  char parola[] = "ciao";

  OutFile << a << "\t" << b << std::endl; // Scrive in output i valori delle variabili a e b
  OutFile << parola << std::endl;
  OutFile.close(); // Chiude il file

  int x    = 0;
  double y = 0;
  std::string word;
  // Apre il file in modalita` lettura
  std::ifstream InFile(filename);

  InFile >> x >> y; // Legge i numeri contenuti nel file e li memorizzo nelle variabili x e y
  InFile >> word;
  InFile.close();

  std::cout << "Il file " << filename <<
    " contiene i numeri: " << x << " e " << y << std::endl;
  std::cout << "e la parola: " << word << std::endl;

  return 0;
}
```

Le classi di `I/O` su file di `fstream` sono:

- `ifstream` per **input**
- `ofstream` per **output**
- `fstream` per **input/output**

Dichiara una variabile di tipo file ed apre il file di nome `filename`

Usa l'operatore `<<` per scrivere l'output sul file già aperto (come si fa su terminale con `cout`)

Usa l'operatore `>>` per leggere l'output sul file già aperto (come si fa su terminale con `cin`)

Il file può essere aperto e chiuso più volte in uno stesso programma, ma al termine del programma viene chiuso automaticamente

Lettura e scrittura da file

Osservazione 1: i costruttori `ifstream` / `ofstream` / `fstream` prendono in input un puntatore a `char`:

```
int main (int argc, char** argv)
{
  // Verifica che venga passato almeno un parametro da riga di comando
  if (argc < 2)
    {
      std::cout << "Digitare il nome del file da riga di comando"
                << std::endl;
      std::cout << "\t./esempio02 File.txt" << std::endl;
      return 1;
    }

  // Usa un oggetto std::string per memorizzare il nome del file
  std::string NomeFile = argv[1];

  std::ofstream OutFile(NomeFile.c_str());
  ...
  std::ifstream InFile(argv[1]);
  ...
}
```

Usa il metodo **`c_str()`** per convertire l'oggetto `std::string` in un puntatore a `char`

Si può anche usare direttamente `argv[1]` come argomento dato che è un puntatore a `char`

Lettura e scrittura da file

Osservazione 2: è possibile usare il metodo `open("NomeFile.txt", OpenMode)` di `fstream` per specificare il tipo di operazione da fare sul file attraverso l'opzione `OpenMode`

```
#include <iostream>
#include <fstream>
int main () {
  std::fstream OutFile;
  OutFile.open("file.txt",std::ios::out);
  OutFile << "12 24" << std::endl;
  OutFile.close();

  int a,b;
  std::fstream InFile;
  InFile.open("file.txt",std::ios::in);
  InFile >> a >> b;
  InFile.close();

  std::cout << "Ho letto: " << a << "\t" << b << std::endl;
  return 0;
}
```

File aperto in sola scrittura
In `std::ofstream` l'`OpenMode` è `std::ios::out` per default

`std::ios::app` si usa per scrivere su un file in modalità *append* (cioè aggiunge righe a quelle già esistenti)

File aperto in sola lettura
In `std::ifstream` l'`OpenMode` è `std::ios::in` per default

Se un file ha molte righe...

```
#include <iostream>
#include <fstream>
#include <string>

int main (int argc, char** argv)
{
  // Verifica che venga passato almeno un parametro da riga di comando
  if (argc < 2)
    {
      std::cout << "Digitare il nome del file da riga di comando"
                << std::endl;
      std::cout << "\t./esempio03 File.txt" << std::endl;
      return 1;
    }
  std::string NomeFile = argv[1];
  std::ofstream OutFile(NomeFile.c_str());
  // Scrittura di piu` righe (std::endl interrompe la riga)
  for (int i = 0; i < 10; i++)
    {
      OutFile << i*2 << "\t" << i*2+1 << std::endl;
    }
  OutFile.close();
...
```

Se un file ha molte righe...

```
...
  int x      = 0;
  int y      = 0;
  int Nrighe = 0;
  std::ifstream InFile(NomeFile.c_str());

  // Imposta un loop infinito interrotto con l'istruzione break
  while (true)
    {
      InFile >> x >> y;

      // Il metodo eof() restituisce true alla fine del file
      if (InFile.eof() == true)
        break;

      std::cout << x << "\t" << y << std::endl;
      Nrighe++;
    }

  InFile.close();
  std::cout << "Il file " << NomeFile << " contiene "
            << Nrighe << " righe" << std::endl;
  return 0;
}
```

Esempio di contenuto del file

```
0        1
2        3
4        5
...      ...
```

Se un file ha molte righe...

N.B.: il seguente codice non funziona (stampa a schermo 2 volte l'ultima riga del file):

```
...
while (InFile.eof() == false)
  {
    InFile >> x >> y;
    std::cout << x << "\t" << y << std::endl;
    Nrighe++;
  }
...
```

Il motivo è che per far diventare `InFile.eof() == true` è necessario effettuare l'operazione di lettura anche dopo aver letto l'ultima riga del file. E` un po' come se il file avesse `N+1` righe (dove `N` è il numero di coppie `x,y` salvate nel file), cioè l'ultima vera riga del file, la `N+1`-esima, contenente il carattere di "`EOF`" (i.e. `End Of File`)

Dopo ogni riga del file letta, e prima di manipolare i dati (e.g. stamparli a schermo), è necessario quindi effettuare il controllo di `eof() == true`

Tips & tricks

È buona norma inserire dei **controlli** che verificano l'integrità del file aperto. Questo vale sia quando si effettua una **lettura** da file sia quando si effettua una **scrittura** su file

```
std::ifstream InFile(filename.c_str());
// Se il file non e` presente nella cartella o non e` possibile aprirlo
// stampa un messaggio di errore
if (InFile.good() == false)
{
  std::cout << "Errore! Non e` possibile aprire il file" << std::endl;
  return 1;
}
```

Esempio di lettura di dati e salvataggio in `std::vector`

```
std::vector<int> vec1;
std::vector<int> vec2;
while (true) {
  InFile >> x >> y;
  if (InFile.eof() == true) break;
  vec1.push_back(x);
  vec2.push_back(y);
  Nrighe++;
}
```

N.B.: non serve conoscere a priori il numero dei dati contenuti nel file

Esercizi

Esercizio 1:

- Scrivere un programma che legga una sequenza di misure con il relativo errore salvata nel file di dati `data.txt` e la inserisca in due `std::vector<double>` (uno per le misure e l'altro per gli errori)
- Al termine della lettura del file di input, impostare un ciclo `for` per rileggere i numeri salvati e calcolare la media pesata e l'errore sulla media
- Stampare infine i risultati in un file di output
- Usare `argc` e `argv` per inserire da terminale i nomi dei file di input e di output
- Leggere i dati implementando una funzione con il seguente prototipo (restituisce `false` se la lettura non è andata a buon fine, altrimenti `true`)

```
bool readData(std::string fileName,
              std::vector<double>& valVec,
              std::vector<double>& valErr);
```

Esercizi

Esercizio 2:

- Creare la classe dei vettori geometrici (2D) nel piano
- Implementare le operazioni con i vettori (somma, sottrazione, prodotto, etc...) secondo l'header file fornito (`vett2d.h`)
- Scrivere inoltre un `main` program in cui vengono testate tutte le funzionalità della classe

Approfondimenti

- Algoritmi di *sorting*
- Smart pointers
- `std::stringstream`
- L'attributo `auto`
- *Function* pointer
- Funtori (i.e. l'overloading dell'operatore `()`)

Algoritmi di *sorting*

Data una serie di dati dello stesso tipo `[a, d, c, ...]` e un criterio che definisce quando un dato sia **>** di un altro, un algoritmo di ordinamento (i.e. *sorting*) è una serie di istruzioni che ordinano la sequenza `[a, c, d, ...]`

Parametri che definiscono le performance di un algoritmo di ordinamento:

- **Rapidità** (i.e. tempo impiegato ad eseguire l'ordinamento): può essere definita per il caso ideale (serie quasi ordinata), per il caso peggiore (serie massimamente disordinata), o per un caso tipico
- **Complessità spaziale** (i.e. quantità di memoria utilizzata): è infatti importante sapere anche quanta memoria viene usata in funzione delle dimensioni della serie, in modo da evitare problemi di utilizzo di memoria
- **Stabilità**: l'algoritmo può comportarsi in maniera diversa, a parità di condizioni, in casi particolari. Tipicamente nel trattare dati dello stesso valore, data la serie `[7,9,2,5,5]`, può essere ordinata come `[2,5,5,7,9]`, oppure come `[2,5,5,7,9]`

Algoritmi di *sorting*

Esistono vari algoritmi di ordinamento che sono stati studiati in dettaglio e classificati in base ai parametri precedentemente citati

Algorithm	Best case	Worst case	Average case	Space complexity	Stable
Merge sort	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	Yes
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Quick sort	$O(n\log n)$	$O(n^2)$	$O(n\log n)$	$\log n$ best, n avg	No
Heap sort	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	No
Counting sort	$O(k+n)$	$O(k+n)$	$O(k+n)$	$O(k+n)$	Yes

Dove n è il numero di dati della serie da ordinare

Gli algoritmi di *sorting* possono essere suddivisi in due categorie principali:

- **Algoritmi a conteggio:** algoritmi di ordinamento specifici per valori numerici interi
- **Algoritmi a confronto:** esaminano gli elementi di una serie mediante una singola operazione di comparazione astratta per determinare di una coppia di elementi quale deve venir posizionato prima nella serie finale ordinata

L'algoritmo *bubble sort*

Bubble sort è un algoritmo a confronto. L'algoritmo scorre la serie varie volte, e ogni volta "porta" l'elemento più grande in fondo alla serie. Diciamo che la serie possiede `n` elementi, alla prima passata della serie l'algoritmo porterà nella posizione `n-1` l'elemento in assoluto più grande. Alla seconda passata porterà il secondo elemento più grande nella posizione `n-2`. Alla terza passata porterà il terzo elemento più grande nella posizione `n-3`, e così via

L'algoritmo possiede quindi:

- un ciclo esterno con indice `i` da `1` a tutti gli elementi della serie
- un ciclo interno con indice `j` da `0` al numero di elementi della serie meno `i`
- nel ciclo interno viene implementato lo spostamento dell'elemento più grande

Si può dire che come "come una bolla" l'elemento più grande risale la serie fino ad essere collocato nella giusta posizione

La soluzione dell'*esercizio 6* della *Lezione 2* è stato implementato con l'algoritmo *bubble sort*

L'algoritmo *quick sort*

Quick sort è un algoritmo a confronto. L'algoritmo si basa sulla ripartizione ricorsiva della serie in base ad una quantità pivotale. In altre parole, scelto un elemento della serie, detto pivot, sposta tutti gli elementi più piccoli a sinistra del pivot, e tutti gli elementi più grandi a destra del pivot. L'algoritmo richiama poi se stesso: una prima volta sulla sotto serie di destra, e una seconda volta sulla sotto serie di sinistra

Il prototipo della funzione potrebbe essere:

```
void QuickSort(double* myArray, int lo, int hi)
```

L'algoritmo possiede quindi:

- un `if` in cui si chiede se `lo < hi`
- come elemento pivot si sceglie `myArray[hi]`
- un ciclo da `lo` a `hi-1` in cui sposta gli elementi più piccoli del pivot e li posiziona a sinistra nella serie (i.e. nella parte bassa della serie)
- alla fine del ciclo sposta il pivot dalla posizione `hi` alla posizione immediatamente successiva a quella occupata dagli elementi più piccoli
- richiama se stesso:

```
QuickSort(myArray, lo, pivot_position - 1);
QuickSort(myArray, pivot_position + 1, hi);
```

L'algoritmo *quick sort*

L'algoritmo descritto in pseudo codice:

```
void QuickSort(double* myArray, int lo, int hi)
  if (lo < hi)
      double pivot = myArray[hi];
      int i = lo;
      for (int j = lo; j < hi; j++)
        if (myArray[j] < pivot)
            if (i != j) swap myArray[j] and myArray[i];
            i++;
  swap myArray[i] and myArray[hi];
```

La STL mette a disposizione delle classi per eseguire l'ordinamento di vettori

Gli algoritmi di *sorting* si trovano nella libreria **<algorithm>**, nella quale si ha a disposizione **std::sort(...)** che è fortemente ottimizzato ed è un ibrido tra *quick sort*, *heap sort* e *insertion sort*, e ma non è stabile, oppure si può usare `std::stable_sort()` che è basato sul *merge sort*. Esempio:

```
std::vector<double> vec;
for (int i = 0; i < DIM; i++) vec.push_back(DIM-i);
std::sort(vec.begin(), vec.end());
```

Forniscono inizio e fine del vettore

Esercizi

<u>Esercizio</u>: Aggiungere all'*esercizio 6* della *Lezione 2* sia l'algoritmo di ordinamento *quick sort*, sia l'ordinamento tramite `std::sort()`. Effettuare un *benchmark* delle prestazioni in termini di tempo di esecuzione (*cfr. esempio* `testPow.cpp` *Lezione 1*)

Smart pointer

Si è visto che nel caso di allocazione dinamica della memoria (cioè usando l'operatore `new`) è necessario ricorrere all'istruzione `delete` per liberare la memoria allocata ed evitare quindi di generare *garbage* nello *heap*

Il `C++` mette a disposizione uno strumento di *garbage collection* che evita la necessità di tener traccia dei `new`: gli **smart pointer**. Con gli smart pointer la memoria viene liberata non appena non è più referenziata, cioè non appena non possiede più un puntatore con il suo indirizzo

Gli smart pointer sono dei **template** e fanno parte della **STL**. Ne vengono fornite due tipologie gli **shared pointer** (**`shared_ptr`**) e gli **unique pointer** (**`unique_ptr`**) entrambi presenti nella libreria **`memory.h`**

- **`shared_ptr`**: può condividere l'oggetto a cui punta con altri **shared pointer**, quindi quando si modifica l'oggetto lo si modifica per tutti gli **shared pointer**
- **`unique_ptr`**: una volta istanziato uno **unique pointer** nessun altro puntatore può accedere all'oggetto a cui punta

Uso degli shared_ptr

```
#include <iostream>
#include <vector>
#include <memory>
#define N 10

void Print(std::shared_ptr<double> s1, std::shared_ptr<double> s2)
{
  std::cout << "\nFirst shared_ptr points to the memory cell : " << s1
            << " which containts the value: " << *s1 << std::endl;
  std::cout <<  "Second shared_ptr points to the memory cell : " << s2
            << " which containts the value: " << *s2 << std::endl;
}

template<typename T>
std::shared_ptr< std::vector<T> > create()
{
  return std::make_shared< std::vector<T> >();
}

void makeVectorAndDestroy()
{
  // Alternative way to instantiate a shared_ptr
  std::shared_ptr< std::vector<double> > sv1 = create<double>();
  sv1->push_back(3.8);
  sv1->push_back(4.2);
  std::cout << std::endl;
...
```

Modo alternativo per inizializzare uno `std::shared_ptr`

Uso degli shared_ptr

```
...
  for (unsigned int i = 0; i < sv1->size(); i++)
    std::cout << "Element [" << i << "] of the vector: " << (*sv1)[i] << std::endl;
}

int main()
{
  std::shared_ptr<double> s1(new double);
  std::shared_ptr<double> s2(new double);
  std::shared_ptr<double> s3 = s1;

  double* myDouble = new double;
  /* Compilation error: it's not possible to assign
     a standard pointer to a shared pointer
     std::shared_ptr<double> s4 = myDouble;
  */

  *s1 = 0.2;
  *s2 = 1.4;
  Print(s1,s2);
  s2 = s1;
  Print(s1,s2);
  *s1 = 2.6;
  Print(s1,s2);

  makeVectorAndDestroy();
...
```

Siccome `sv1` è un puntatore ad un `std::vector`, bisogna prima prendere il contenuto puntato da `sv1` e poi applicare l'`operator[]`

Modo tradizionale per inizializzare uno `std::shared_ptr`

Alla chiusura dello scope di **`makeVectorAndDestroy`** lo **smart pointer** `sv1` (creato nello *heap*) viene automaticamente distrutto e la memoria quindi liberata
Il programmatore non deve più preoccuparsi di usare l'istruzione `delete`

Uso degli shared_ptr

```
...
  // Test with arrays
  std::cout << "\nShared pointers and arrays:" << std::endl;
  // We need to provide a custom delete because otherwise by defaut shared pointers
  // call delete var_name; but for arrays one needs to call delete[] var_name;
  std::shared_ptr<double> myArray(new double[N], std::default_delete<double[]>());

  for (unsigned int i = 0; i < N; i++)
    myArray.get()[i] = static_cast<double>(i) / N;
    // get() retrieves the standard pointer

  for (unsigned int i = 0; i < N; i++)
    std::cout << "Element [" << i << "] = "
              << myArray.get()[i] << std::endl;

  return 0;
}
```

Per accedere al singolo elemento dell'array bisogna prima chiedere il puntatore standard all'array tramite l'istruzione **get()** e poi usare l'`operator[]`

Solo dal `C++17` gli **shared pointer** supportano la creazione di array. Per versioni antecedenti è necessario fornire il distruttore appropriato, infatti di *default* gli **shared pointer** chiamano la `delete var_name;` invece per gli array è necessario chiamare la `delete[] var_name;`

Dal `C++17` gli array sono completamente supportati dagli **shared pointer**

- Creazione: `std::shared_ptr<double[]> myArray(new double[N]);`
- Accesso agli elementi tramite l'`operator[]` (i.e. `myArray[i]`)

Uso degli unique_ptr

```
#include <iostream>
#include <vector>
#include <memory>
#define N 10

void Print(std::unique_ptr<double>& u1, std::unique_ptr<double>& u2)
{
  std::cout << "\nFirst unique_ptr points to the memory cell : " << u1
            << " which containts the value: " << *u1 << std::endl;
  std::cout <<  "Second unique_ptr points to the memory cell : " << u2
            << " which containts the value: " << *u2 << std::endl;
}


template<typename T>
std::unique_ptr< std::vector<T> > create()
{
  std::unique_ptr< std::vector<T> > uv(new std::vector<T>());
  return std::move(uv);
}

void makeVectorAndDestroy()
{
  // Alternative way to instantiate a unique_ptr
  std::unique_ptr< std::vector<double> > uv1 = create<double>();
  uv1->push_back(3.8);
...
```

Dal C++14 è possibile usare `std::unique_ptr`:
`return std::make_unique< std::vector<T> >();`
Proprio come per gli `std::shared_ptr`

Nel caso degli **unique pointer** non è possibile che due puntatori puntino alla stessa area di memoria (i.e. non è consentita la copia, **=**)

Esiste quindi l'istruzione **`move(...)`** per "passare" l'indirizzo di un'area di memoria da un puntatore ad un altro

Uso degli `unique_ptr`

```
...
  uv1->push_back(4.2);
  std::cout << std::endl;
  for (unsigned int i = 0; i < uv1->size(); i++)
    std::cout << "Element [" << i << "] of the vector: " << (*uv1)[i]
              << std::endl;
}

int main()
{
  std::unique_ptr<double> u1(new double);
  /* Unique pointers can not be copied in any way
     These two lines give compilation error
     std::unique_ptr<double> u2(u1);  // Copy constructor
     std::unique_ptr<double> u3 = u1; // Copy assignment
  */
  std::unique_ptr<double> u4(new double);

  *u1 = 0.2;
  *u4 = 1.4;
  Print(u1,u4);


  // u4 = u1; // This is not permitted with unique_ptr
  u4 = std::move(u1); // At this point u1 is empty, so it can not be used,
                      // we need to reset it
...
```

Siccome `uv1` è un puntatore ad un `std::vector`, bisogna prima prendere il contenuto puntato da `uv1` e poi applicare l'`operator[]`

Alla chiusura dello scope di **`makeVectorAndDestroy`** lo **smart pointer** `uv1` (creato nello *heap*) viene automaticamente distrutto e la memoria quindi liberata. Il programmatore non deve più preoccuparsi di usare l'istruzione `delete`

Uso degli `unique_ptr`

```
...
  u1.reset(new double(2.6));
  Print(u1,u4);

  u1.reset(new double(3.8));
  u4.reset(new double(4.2));
  Print(u1,u4);

  makeVectorAndDestroy();

  // Test with arrays
  std::cout << "\nUnique pointers and arrays:" << std::endl;
  // No need to provide a custom delete because unique pointers already support
  // dynamical allocation of arrays
  std::unique_ptr<double[]> myArray(new double[N]);

  for (unsigned int i = 0; i < N; i++)
    myArray[i] = static_cast<double>(i) / N;
    // get() retrieve the standard pointer

  for (unsigned int i = 0; i < N; i++)
    std::cout << "Element [" << i << "] = " << myArray[i] << std::endl;

  return 0;
}
```

L'istruzione **`reset(...)`** consente di cambiare l'area di memoria puntata dal puntatore cancellando automaticamente quella non più referenziata

Sin dal `C++98` gli **unique pointer** supportano la creazione di array, è infatti possibile usare anche la sintassi **`[]`**

Esercizi

Esercizio 1: Trovare e correggere gli errori riguardanti l'uso degli **smart pointer**

```
#include <iostream>
#include <memory>
#include <vector>

int main()
{
  std::shared_ptr<double> s1;
  std::shared_ptr<double> s2;
  s2 = s1;
  std::cout << "s1 contains: " << *s1 << "; s2 contains: " << *s2
            << std::endl;

  std::shared_ptr<std::vector<double>> s3(new std::vector<double>());
  s3->push_back(1.2);
  std::cout << "s3[0] contains: " << s3->operator[](0) << std::endl;

  std::shared_ptr<double[]> sa1(new double[10]);
...
```

Esercizi

```
...
  std::shared_ptr<double> sa2;
  sa2 = std::make_shared<new double[10]>();

  std::unique_ptr<double> u1;
  std::unique_ptr<double> u2;
  u2 = u1;
  std::cout << "u1 contains: " << *u1 << "; u2 contains: " << *u2
            << std::endl;

  double* d = new double(1.2);
  std::unique_ptr<double> u1;
  u1 = d;

  std::unique_ptr<double> ua1(new double[10]);
  ua1[0] = 1.2;

  return 0;
}
```

Esercizi

Esercizio 2: Ripetere l'*esercizio 1* della *Lezione 5* utilizzando solo `std::unique_ptr` nella classe `SimpleArray`

Esercizio 3: Ripetere l'*esercizio 1* della *Lezione 5* utilizzando solo `std::shared_ptr` nella classe `SimpleArray`

std::stringstream

Gli oggetti di tipo `std::stringstream` hanno a disposizione dei metodi molto utili per manipolare flussi di dati, potendo gestire sia numeri che parole in maniera analoga ai metodi `std::cin` e `std::cout`

```
#include <iostream>
#include <sstream>

using namespace std;
int main ()
{
  stringstream ss;
  int anno    = 1923;
  string nome = "John";

  // Posso unire numeri e stringhe in un unico stream
  ss << nome << " nacque nel " << anno;
  // Conversione sstream -> string
  string frase = ss.str();
  cout << frase << endl;

  return 0;
}
```

N.B.: per svuotare una `std::stringstream` è necessario utilizzare la coppia di metodi:

```
ss.clear();
ss.str("");
```

Si scrive nella variabile `std::stringstream` con l'operatore << (come con `std::cout`)

Lettura file con stringstream

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
using namespace std;
int main(int argc, char** argv)
{
  string filename = argv[1];
  ifstream InFile (filename.c_str());
  // Metodo good() per verificare che sia possibile leggere il file
  if (InFile.good() == false)
    {
      cout << "Errore! Non è possibile aprire il file " << filename << endl;
      return 1;
    }
  // Usa il metodo getline (istream& is, string& str)
  // per leggere il file una riga alla volta
  // (restituisce falso alla fine del file)
  string line, parola;
  stringstream ss;
  int nRighe = 0;
  while (getline(InFile, line))
    {
      nRighe++;
      cout << "\nRiga " << nRighe << ": " << line << endl;
      cout << "Selezione parole pari: ";

      int nParole = 0;
      ss << line; // Riempie lo stringstream con la riga
      while (ss >> parola) // Legge parola per parola
            {
              nParole ++ ;
              if ((nParole%2) == 0)
                cout << parola << " ";
              else continue;
            }
      cout << endl;
      ss.clear(); ss.str(""); // Svuota lo stringstream
    }
  return 0;
}
```

Una variabile `std::stringstream` può essere usata per `I/O` da file

getline prende in input un oggetto `ifstream` (file da leggere) e un oggetto `std::string`, in cui memorizzare la riga del file. Restituisce **false** alla fine del file

Con l'operatore **>>** viene letto lo stream come si farebbe con `std::cin`

In questo esempio si è deciso di selezionare solo le parole che occupano una posizione pari all'interno della riga

L'attributo auto

L'attributo auto (introdotto dal c++11 in poi) consente di dichiarare delle variabili di tipo non specificato perché il tipo verrà automaticamente dedotto a *compile time*. Inoltre può essere usato nelle funzioni che fanno uso di return per "alleggerire" la sintassi:

```
auto a = 1 + 2; // Ad a viene assegnato il tipo int
auto b = 1 + 1.2; // A b viene assegnato il tipo double
auto x; // Fornisce errore
for (auto i=0; i<10; i++) // Ad i viene assegnato il tipo int
auto myFunction(...) -> decltype ( ... )
{
  ...
  return val;
}
```

Dal C++14 l'uso dell'attributo auto è stato esteso ad altri casi

Esercizio: Usare nell'esercizio 1 l'attributo auto dove possibile

L'attributo auto

Esempio di uso di **auto** per identificare il tipo associato al `return`:

```
#include <iostream>
#define N 100
int makeObject()
{
  return N;
}
auto testReturn() -> decltype ( makeObject() )
{
  auto val = makeObject();
  std::cout << "I use the new type: " << val << std::endl;
  return val;
}
int main ()
{
  testReturn();
  return 0;
}
```

Notare anche questo elemento sintattico: **decltype (...)** consente di ottenere il tipo di ciò che sta tra parentesi. Necessario nel `c++11` per determinare il tipo da assegnare a `val`, ma a partire dal `C++14` risulta superfluo

Lo stesso programma, infatti, in `c++14` può essere scritto con questa semplice modifica:

auto `testReturn()`

al posto di

auto `testReturn()` **-> decltype** **(** `makeObject()` **)**

Tramite l'uso congiunto di **auto** e **decltype** non si è dovuto specificare il tipo di `val` e nemmeno del `return`

L'attributo auto

Inoltre la potenza di **auto** la si apprezza anche nell'uso dei cicli `for` dove non è più necessario specificare il tipo dell'elemento dell'array su cui si itera. Ecco un esempio di **"iteratore universale"** che fa uso dell'attributo **auto** e dei template:

```
...
template<typename T>
void myGenericIterator (T& myIterable) {
  for (auto ele : myIterable)
    std::cout << ele << std::endl;
}
int main()
{
  std::string myArray[] = {"Giulia", "Alessandro", "Pietro"};
  std::vector<std::string> myVec;
  myVec.push_back(myArray[0]);
  myVec.push_back(myArray[1]);
  myVec.push_back(myArray[2]);
  myGenericIterator(myVec);
  std::cout << std::endl;
  myGenericIterator(myArray);
  return 0;
}
```

Nuova sintassi del ciclo `for`: itera automaticamente sugli elementi di `myIterable` assegnando ogni volta ad `ele` un valore diverso

Notare che è stata omessa la specifica del tipo: `function_name<type>(...)` perché risulta superflua quando, come in questo caso, il tipo può essere dedotto a *compile time.* (esempio di *template argument deduction*) **N.B.: Function** template argument deduction dal `c++98`, mentre **class** template argument deduction dal `c++17`

Function pointer

Il concetto di puntatore può essere esteso anche alle funzioni. E` infatti possibile creare puntatori a funzioni. La sintassi è la seguente:

```
int raddoppia (int a); // Prototipo della funzione a cui si vuol far puntare il function pointer (queste funzioni sono anche dette callback function)
...
int (*nome_puntatore) (int); // Puntatore ad una funzione che restituisce un int e che in input vuole un int
...
nome_puntatore = &raddoppia; // Ora il puntatore punta alla funzione raddoppia
...
std::cout << nome_puntatore(2) << std::endl; // Chiamata alla funzione raddoppia
```

Sintassi generale di un *function* pointer:

```
tipo_output (*nome_puntatore) (lista_tipi_input);
```

Come si interpreta questo function pointer? `int*(*foo)(int*);`

Function pointer: esempio

Si hanno due funzioni di *sorting* (*quick sort* e *bubble sort*). Si vuole creare un *function* pointer che punti a una o all'altra a seconda della scelta dell'utilizzatore. Il codice che esegue la chiamata deve invece essere indipendente dalla scelta dell'utilizzatore:

```
void BubbleSort(double* myArray, int dim, int tmp = 0);
void QuickSort(double* myArray, int lo, int hi);
void GeneralSort(void (*sortAlgo) (double*, int, int), double*
myArray, int i1, int i2);
...
void (*sortAlgo) (double*, int, int);
...
sortAlgo = &BubbleSort;
...
GeneralSort(sortAlgo,data,i1,i2);
```

Esercizio: Creare un programma che faccia uso della `GeneralSort` per lanciare l'algoritmo di *sorting* scelto dall'utente

La funzione `GeneralSort` riceve in ingresso sia un *function* pointer che punterà all'algoritmo di *sorting* scelto dall'utente, che i relativi parametri da passare all'algoritmo di *sorting*

- Istanziazione del *function* pointer
- Scelta dell'algoritmo di *sorting*
- Chiamata all'algoritmo di *sorting* che non dipende dal tipo di algoritmo

Funtori (i.e. l'overloading dell'operatore ( ))

L'overloading dell'operatore ( ), detto operatore di chiamata di funzione, è un interessante caso di studio. Grazie ad esso è possibile implementare un costrutto comunemente impiegato nella programmazione orientata agli oggetti: il **funtore** (*functor*)

In C++, infatti, ogni istanza di una classe dotata dell'overload dell'operatore ( ) è detta **funtore**. Il nome deriva dal fatto che l'invocazione di tale operatore è praticamente indistinguibile dalla sintassi adottata per l'invocazione di una funzione

I **funtori** sono comunemente impiegati in C++ per l'implementazione di *callback*, in sostituzione dei puntatori a funzione tradizionalmente in uso nel linguaggio C

Funtori (i.e. l'overloading dell'operatore ( ))

Vediamo ora un semplice esempio di uso dei **funtori**:

```
class QuickSort
{
public:
  void operator() (double* myArray, int lo, int tmp = 0)
  {
  // Implementazione dell'algoritmo di quick sort
  }
};

class BubbleSort
{
public:
  void operator() (double* myArray, int lo, int hi)
  {
  // Implementazione dell'algoritmo di bubble sort
  }
};
```

Funtori (i.e. l'overloading dell'operatore ( ))

```
template <typename T>
void GeneralSort (T sortFunctor,
                  double* myArray,
                  int i1,
                  int i2)
{
  sortFunctor(myArray,i1,i2);
}

int main()
{
  ...
  // Chiamata all'algoritmo bubble sort
  BubbleSort bs;
  bs(data, DIM);
  // Chiamata all'algoritmo quick sort
  QuickSort qs;
  qs(data, 0, DIM-1);
  ...
```

Funtori (i.e. l'overloading dell'operatore ( ))

```
   ...
   // Chiamata tramite la funzione GeneralSort
   // Sfrutta la template argument deduction
   GeneralSort(BubbleSort(), data, i1, i2);
   GeneralSort(QuickSort(), data, i1, i2);
   ...
}
```

Istanzia l'oggetto **BubbleSort** (**QuickSort**)

Con questa sintassi viene invocato il default constructor, infatti in questo contesto non c'è ambiguità con la definizione di una funziona (*cfr. Lezione 4*)

L'uso dei **funtori** presenta notevoli vantaggi, sia a *compile time* che a *run time*. In primo luogo perché la sintassi usata per la definizione dei *function pointer* non è facilmente leggibile, inoltre, un *function pointer* può essere facilmente riassegnato a *run time*, pertanto raramente il compilatore applicherà delle ottimizzazioni alla funzione puntata

Un ulteriore aspetto interessante dei funtori è che possono contenere dei parametri poiché sono delle classi. Per esempio se l'algoritmo di *quick sort* avesse avuto bisogno di un

Funtori (i.e. l'overloading dell'operatore ( ))

parametro, avremmo potuto modificare il codice in questo modo:

```
class QuickSort
{
public:
  QuickSort(double par) : parameter(par) {}
  void operator() (double* myArray, int lo, int tmp = 0)
  {
  // Implementazione dell'algoritmo di quick sort che fa
  // uso anche del parametro "parameter"
  }
private:
  double parameter;
};
```

Nel main program:

```
GeneralSort(BubbleSort(3.141), data, i1, i2);
```

Esercizio: Modificare l'esercizio a *pagina 41* di *questa Lezione* in maniera tale da usare i **funtori** al posto dei *function pointer*